Qwen

From World Space to Screen Space: A Compiler-Style Approach to Defining Spatial and Perceptual Interfaces in Semantic Runtimes

The Foundational Dichotomy: Separating Objective Existence from Observer-Relative Perception

The core architectural challenge in complex semantic runtime systems is managing the inherent complexity of three-dimensional environments. As systems grow to encompass diverse functionalities such as navigation, physics simulation, autonomous AI reasoning, and high-fidelity rendering, a critical risk emerges: the contamination of state. This occurs when modules designed for fundamentally different purposes—such as calculating collision (physics) and determining what to draw on screen (rendering)—begin to share or conflate their understanding of spatial reality. The research goal of formalizing a strict boundary between Coordinate Application Binary Interface (ABI), which defines objective spatial existence, and View Address ABI, which defines observer-relative perception, directly addresses this challenge. This separation is not merely a novel engineering pattern but a principled architecture grounded in decades of research across computer graphics, robotics, geographic information systems (GIS), and cognitive science. It establishes a clear distinction between what exists in the world and how it appears to an observer, thereby providing a stable foundation upon which interdependent systems can be built without mutual coupling. The central thesis is that by enforcing this dichotomy, the system gains robustness, scalability, and maintainability, as each module interacts with precisely the information it needs, and only that information.

This proposed separation finds its most direct parallel in the foundational concepts of computer graphics, specifically the distinction between World Space and Screen Space [1][2]. World Space represents an objective, shared coordinate system where all objects are placed with absolute positions and orientations, independent of any camera or observer [1][2]. It is the canonical space for building scenes and reasoning about object relationships, including those used by physics engines for collision detection and by navigational algorithms for pathfinding. Conflating World Space with other spaces, such as attempting

to perform lighting calculations in Screen Space, leads to incorrect results and difficult-to-diagnose bugs [12]. The proposed CoordinateABI serves as the formal specification for an object's state within this invariant World Space. Conversely, Screen Space is relative to the camera's perspective, dealing with pixels, viewport coordinates, and depth values based on what the camera can actually see [12]. The process of transforming geometry from World Space through various intermediate spaces (like Clip Space and Normalized Device Coordinates) to arrive at Screen Space is a well-understood pipeline [12]. The ViewAddressABI can be seen as a high-level abstraction of this transformation process, capturing the essential perceptual attributes needed by downstream modules like the renderer, without exposing them to the low-level mechanics of projection matrices or vertex shaders. This aligns with modern graphics API design philosophies, such as Vulkan, which provide explicit control over GPU operations by minimizing error checking and requiring developers to manage these transformations manually, thus reinforcing the need for clear conceptual boundaries [17] [19].

The principles of separating spatial truth from perceptual truth are also deeply embedded in robotics and simulation frameworks, particularly the Robot Operating System (ROS). The ROS TF2 (Transform) library is explicitly designed to manage the relationships between multiple coordinate frames over time [23] [24]. It maintains a tree structure of transforms, allowing any frame to query its position and orientation relative to any other frame in the tree [57]. For instance, a robot's base link can have a known ground-truth pose in the world frame, while a sensor mounted on it might report a pose relative to the base link [25]. This system inherently distinguishes between the robot's true, often estimated, ground-truth position and its sensor-derived perceptions [26] [32]. The discrepancy between simulated states and standard ROS TF structures highlights the importance of maintaining a consistent reference frame [56]. In this context, the CoordinateABI would correspond to an object's definitive pose within a fixed world frame, analogous to a robot's ground-truth pose [26]. The ViewAddressABI would then be derived by transforming the CoordinateABI into the local coordinate frame of an observer (e.g., a camera), effectively performing the kind of transform calculation that TF2 excels at. This ensures that a perception module processing visual data understands the object's location relative to the camera, while a planning module continues to reason about the object's absolute location in the world, preventing contamination of the planning logic with sensor-specific noise or uncertainties.

Furthermore, this architectural dichotomy is supported by theoretical models in Geographic Information Systems (GIS). Standards such as ISO 19107 define a conceptual schema for describing the spatial characteristics of geographic entities and the topological relations between them [9] [40]. These schemas provide a way to describe *what* is located

where, establishing a spatial ontology that is independent of any particular visualization or rendering method [49] [50]. International standards like CityGML serve as a concrete example, providing a data model for storing and sharing 3D city models that separates geometric descriptions from rich semantic content [27] [46] [47]. A building's geometry and ownership are distinct properties; one can change (e.g., adding a new facade) without necessarily altering the other. This mirrors the proposed separation between CoordinateABI (defining the building's location and ownership) and ViewAddressABI (defining which face of the building is visible from a given viewpoint). The ability to reason about qualitative spatial relations, such as adjacency or containment, is a core function of GIS and relies on a stable, underlying representation of spatial truth [9]. By formalizing CoordinateABI and ViewAddressABI, the semantic runtime system adopts a similar philosophy, ensuring that its internal model of the world remains consistent and reliable, regardless of how it is perceived or rendered.

Finally, the distinction between an object's intrinsic identity and its view-dependent appearance is a cornerstone of human and artificial spatial cognition. Cognitive science research distinguishes between object-centric and view-centric representations [13]. Object-centric models build a stable, holistic 3D representation of an object by integrating information from multiple viewpoints, preserving the object's identity across changes in perspective. In contrast, view-centric representations reflect the spatial organization of a scene tied directly to the observer's position and orientation, producing content that is highly context-dependent [13]. Tasks that require mental rotation, a key measure of spatial ability, fundamentally rely on the brain's capacity to manipulate an object's view-centric representation while maintaining its object-centric identity [5] [14]. Recent studies on Large Multimodal Models (LMMs) have revealed a significant failure in this area; these models perform poorly when asked to reason about a scene from a 'human perspective' versus a 'camera perspective', indicating they struggle to disentangle the objective state of the world from the observer-relative perception [15]. The proposed architecture directly addresses this challenge by enforcing a clean separation at the software level. The CoordinateABI provides the stable, object-centric anchor, while the ViewAddressABI captures the view-centric information required for rendering and certain types of perceptual reasoning. This principled separation is therefore not just an engineering convenience but a necessary step toward building more robust and cognitively plausible AI and simulation systems. It recognizes that a renderer is not reality and a camera is not truth; rather, perception is a projection of an invariant spatial state [21] [30].

Compiler-Style Interface Specifications: Defining CoordinateABI and ViewAddressABI

To enforce the architectural boundary between spatial truth and perceptual truth, it is essential to move beyond abstract concepts and establish formal, compiler-style interface specifications for both CoordinateABI and ViewAddressABI. These ABIs act as contracts that define the precise set of data fields and properties that a module must provide or consume, thereby guaranteeing a clean separation of concerns. The primary value of this approach lies in defining the boundaries themselves, which protects downstream systems from the complexities and implementation details of upstream modules. This section provides detailed, structured definitions for both ABIs based on the user's requirements and supplemented by relevant contextual knowledge, prioritizing clarity and precision over implementation specifics.

The Coordinate Application Binary Interface (ABI) is responsible for encoding the ground-truth spatial existence of an entity within the semantic runtime. Its properties are absolute, invariant, and independent of any observer. A formal specification for CoordinateABI must include the following core components:

Field Name	Data Type	Description
`world_x`, `world_y`, `world_z`	Floating-point / Integer	The canonical Cartesian coordinates representing the object's origin or representative point in the global world space. This is the single source of truth for the object's location. [12]
`chunk_x`, `chunk_y`, `chunk_z`	Integer	The coordinates of the voxel or tile within a larger grid-based chunking system. This is crucial for efficient memory management, culling, and loading/unloading of terrain. [8]
`elevation`	Floating-point	The height of the object's representative point above a predefined vertical datum (e.g., sea level or world zero). This is vital for terrain rendering, fluid dynamics simulations, and navigation.
`semantic_occupancy`	Enum / String Code	A code or flag indicating the type of content occupying the spatial unit (e.g., `AIR`, `STONE`, `WATER`, `PLAYER`, `NPC`). This is used by physics for collision and by AI for pathfinding. [43] [54]
`spatial_ownership`	Identifier (String/UUID)	An identifier for the entity, player, or system that owns or controls this spatial unit. This is essential for conflict resolution in multi-agent environments.

These fields collectively provide a complete and unambiguous description of an object's presence in the world. Any system requiring knowledge of an object's fundamental location—such as a physics engine for collision detection, a navigation stack for pathfinding, or an AI agent for reasoning about its environment—should depend solely on the CoordinateABI. This prevents them from being coupled to rendering-specific details or camera-related calculations.

In contrast, the View Address ABI is a dynamic interface that captures the perceptual attributes of an object relative to a specific observer. Its state is not intrinsic to the object itself but is a function of the object's CoordinateABI combined with the observer's viewpoint. The formal specification for ViewAddressABI includes the following fields:

Field Name	Data Type	Description
`observer_direction`	Vector3	A normalized vector indicating the direction the observer (e.g., camera, AI eye) is facing. This is the primary input for determining visibility.
`visible_face`	Enum / String Code	A discrete identifier specifying which surface of the object is primarily visible to the observer. Examples include `TOP`, `BOTTOM`, `NORTH`, `SOUTH`, `EAST`, `WEST`, `SLOPE`, `WALL`, `EDGE`, or `UNDERSIDE`.
`normals`	Vector3 Array	One or more surface normal vectors at the point(s) of intersection with the viewer's line of sight. Normals are critical for realistic lighting and shading calculations. [37] [52]
`occlusion`	Boolean / Float	A metric indicating whether the object is fully or partially obscured by other objects in the scene from the observer's viewpoint. This can be pre-calculated using techniques like ambient occlusion maps [6] [7] or determined dynamically.
`render_side`	String Code	A hint for the renderer specifying which texture, material, or mesh variant to use. For example, a `visible_face` of `"EAST"` would map to a `render_side` of `"east_wall_variant"`.
`perspective_relative_address`	String	A compact, dot-separated string representation of the view address (e.g., `u10x.e16y.d56z.north.top`). This field serves as an efficient lookup key for resources.

The ViewAddressABI is generated by a dedicated component that takes an object's immutable CoordinateABI and the current observer's state as inputs. This generation process encapsulates all the logic for determining visibility, such as face culling [51], perspective projection [28], and normal calculation [37]. By presenting this information through a standardized ABI, the rendering pipeline can make informed decisions about which assets to draw without needing to understand the underlying physics or navigation logic. This creates a single point of entry for all perceptual concerns, enforcing the architectural boundary and preventing contamination. For instance, the `tile_address.py` script mentioned in the initial conversation naturally fits within this ViewAddressABI generation process. Its purpose is to translate spatial coordinates, normals, and camera perspective into a compact address, which aligns perfectly with the `perspective_relative_address` field. This demonstrates that `tile_address.py` should consume ground truth and not define it, validating its placement within the ViewAddressABI layer rather than being a core contribution of the research itself. The focus remains on the rigorous definition of the contract (the ABI) that governs this translation.

Dependency Architecture and Data Flow: Visualizing the Boundary Enforcement Mechanism

A successful implementation of the CoordinateABI and ViewAddressABI separation hinges on a clearly defined dependency architecture and a predictable data flow. The goal is to create a system where modules that operate on objective spatial truths are entirely decoupled from modules that deal with observer-relative perceptions. This is achieved by designing a dependency graph where information propagates in a single, well-defined direction: from the invariant CoordinateABI to the variable ViewAddressABI, and finally to the rendering output. This section will outline this dependency structure and provide visual representations to clarify how the architectural boundary is enforced throughout the system.

The dependency diagram for the semantic runtime system can be broken down into two distinct domains separated by the ABI boundary. On one side, we have the core simulation and logic modules that rely exclusively on the CoordinateABI. On the other side, we have the presentation and perception modules that consume the ViewAddressABI. The interaction between these two domains is mediated by a specialized component—the View Address Generator—which acts as a pure function, transforming spatial state into perceptual state.

Domain 1: Core Simulation & Logic (Dependent on CoordinateABI)

This domain contains the fundamental systems that define the rules and state of the virtual world. Their operation must remain independent of any specific observer or rendering technique.

- **Physics Engine:** This module performs all collision detection, rigid body dynamics, and constraint solving. It requires access to the `world_x, world_y, world_z` and spatial extent of objects, which are part of the CoordinateABI. It operates entirely within World Space, making no assumptions about cameras or screens [1][2].
- **Navigation Stack:** Pathfinding algorithms like A* or Dijkstra's operate on a spatial graph or grid. They use the `world_x, world_y, world_z` coordinates to calculate routes and the `semantic_occupancy` data to determine traversable terrain. Their output is a sequence of CoordinateABIs representing a valid path.
- **AI Reasoning Engine:** Autonomous agents use the CoordinateABI to perceive their environment, plan actions, and interact with objects. For example, an AI dragon's decision to fly towards a treasure chest depends on the chest's `world_x, world_y, world_z`. The AI reasons about the chest's *location*, not how it appears from a

specific angle. This aligns with the concept of separating an agent's internal state from its external environment, as seen in frameworks like Agent Behavioral Contracts (ABC) [11].

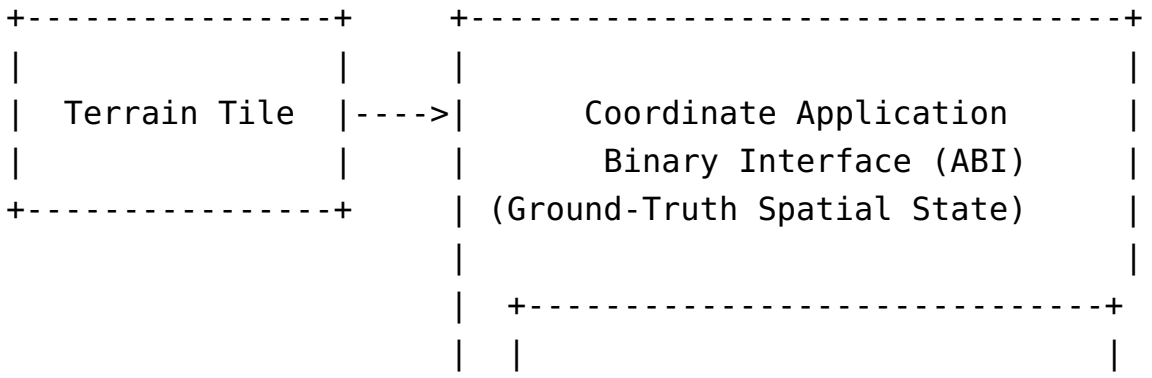
- **Spatial Database / Chunk Manager:** This system manages the storage and retrieval of terrain data. It uses the `chunk_x, chunk_y, chunk_z` coordinates to load and unload data from memory or disk, optimizing performance based on spatial locality [8].

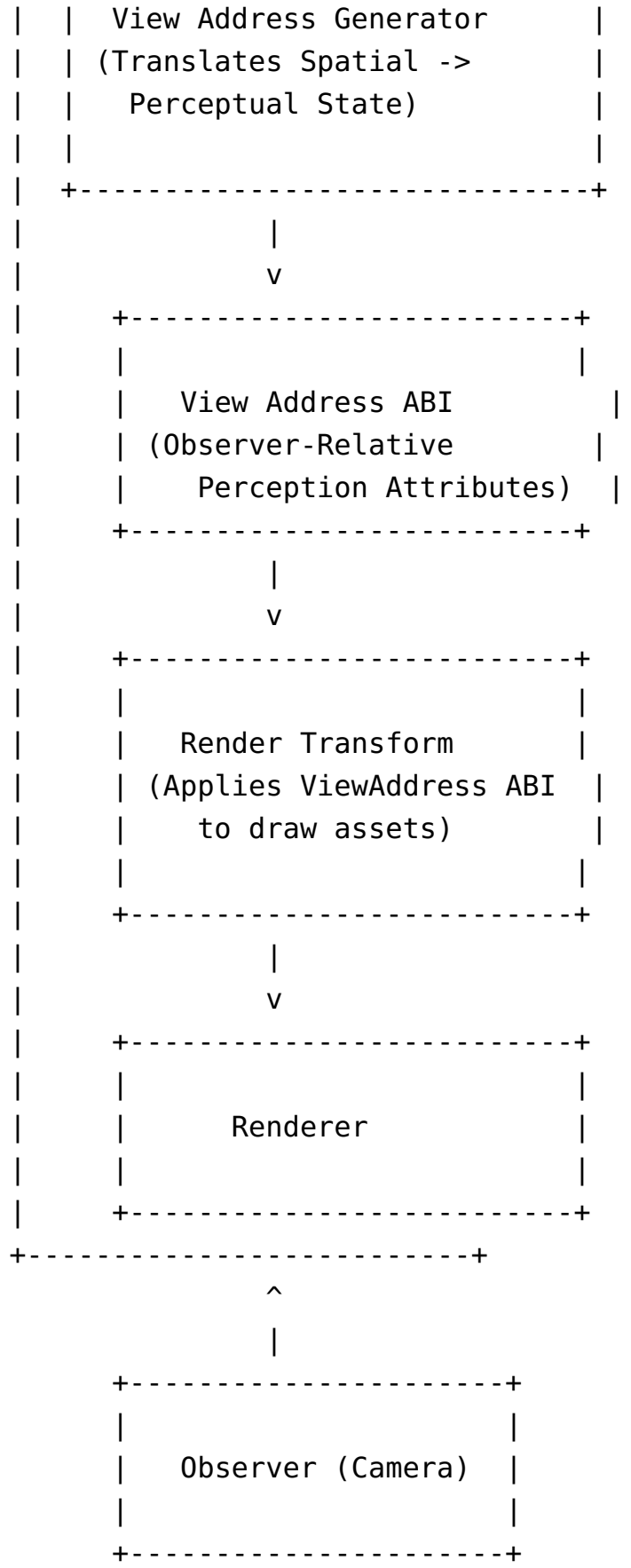
Domain 2: Perception & Rendering (Dependent on ViewAddressABI)

This domain is concerned with creating a believable visual representation of the world for an observer. All modules here are consumers of the ViewAddressABI.

- **View Address Generator:** This is the critical intermediary component. It receives an object's CoordinateABI and an observer's context (e.g., camera transform, direction vector). It then executes the logic to compute the full ViewAddressABI, including the `visible_face`, `normals`, and `render_side`.
- **Renderer / Asset Pipeline:** The final rendering stage consumes the ViewAddressABI. Based on the `render_side` field, it selects the appropriate mesh, texture, and material to draw the object. For a voxel block, if the ViewAddressABI indicates `visible_face` is `"EAST"`, the renderer will draw the east-facing wall variant of the block. This is a direct application of the principle that perception is a projection of spatial state [31].
- **Perception Pipeline (for AI):** While the core AI may reason with CoordinateABI, a perception pipeline could exist to allow an AI to "see" the world. This pipeline would generate a simplified ViewAddressABI for the AI to process, perhaps focusing on visible faces and occluded objects to inform tactical decisions. This mirrors real-world robotic perception pipelines that process sensor data to understand the environment [32] [33].

The dependency flow can be visualized as follows:





This diagram makes the architectural boundary explicit. The arrow from `Terrain Tile` to `CoordinateABI` signifies that the tile's fundamental properties define the ABI's state.

The arrow from `CoordinateABI` to the `View Address Generator` shows that the generator reads the invariant state. The arrow from the generator to `ViewAddressABI` shows the creation of the perceptual state. Finally, the arrow from `ViewAddressABI` to the `Render Transform` shows the consumption of the perceptual state. There are no reverse dependencies. The Physics Engine does not read from ViewAddressABI. The Renderer does not query CoordinateABI directly. The View Address Generator is the sole authority for translating spatial truth into perceptual truth. This structure enforces modularity and prevents the cross-contamination of state that plagues less disciplined architectures. Each module knows exactly what it needs and where to get it, leading to a more robust, testable, and scalable system.

Use-Case Scenarios: Demonstrating Invariant Truth and Variable Perception

The theoretical benefits of separating CoordinateABI and ViewAddressABI are best understood through concrete use-case scenarios that demonstrate the principle of invariant spatial truth versus variable perceptual truth. The core insight—that the world's state never changes, only its perceived representation does—is the key to preventing architectural decay. This section explores several scenarios, starting with the user-provided cliff example, to illustrate how this architecture enables correct behavior across different contexts, from rendering and AI to simulation.

Scenario 1: Cliff Rendering from Multiple Perspectives

This scenario, central to the initial research goal, vividly illustrates the power of the proposed ABIs. Consider a single terrain tile located at the edge of a plateau, possessing a fixed CoordinateABI.

- **Initial State (Invariant):** The terrain tile has a `CoordinateABI` with `world_x=10, world_y=16, world_z=56`. This coordinate is absolute and will never change, regardless of anything else in the system. It is the tile's permanent address in the world.
- **Observer Input 1 (Camera A):** A camera is positioned directly above the tile, looking straight down. Its `observer_direction` is `(0, 1, 0)` (positive Y-axis).
- **View Address Generation:** The View Address Generator processes the tile's CoordinateABI and the camera's direction. It calculates that the primary visible face is the top of the tile.

- **Output (ViewAddressABI):** `visible_face = "top"`, `render_side = "top_variant"`.
- **Rendering Decision:** The renderer uses the `render_side` field to select and draw the flat-top texture for this tile.
- **Observer Input 2 (Camera B):** A second camera is positioned to the east of the cliff, looking west. Its `observer_direction` is `(-1, 0, 0)` (negative X-axis).
- **View Address Generation:** The generator recalculates based on the new viewpoint. It now determines that the primary visible face is the eastern face of the tile.
- **Output (ViewAddressABI):** `visible_face = "east"`, `render_side = "east_wall_variant"`.
- **Rendering Decision:** The renderer selects and draws the vertical-east-wall texture for the same physical tile.
- **Observer Input 3 (Camera C):** A third camera is located inside a cavern beneath the plateau, looking up at the underside of the tile. Its `observer_direction` is `(0, -1, 0)` (negative Y-axis).
- **View Address Generation:** The generator produces a completely different perceptual summary.
- **Output (ViewAddressABI):** `visible_face = "underside"`, `render_side = "underside_variant"`.
- **Rendering Decision:** The renderer draws a dark, cave-like texture for the tile.

This scenario proves the core architectural tenet: a single, immutable CoordinateABI can produce multiple, distinct ViewAddressABIs depending on the observer. The rendering system is simplified because it only needs to react to the `render_side` hint provided by the View Address Generator, offloading all complex visibility logic to a single, centralized component. This is far more robust than having the renderer or other modules try to deduce visibility from raw coordinates and camera frustums, which is a common source of bugs and performance issues [12].

Scenario 2: Autonomous Navigation and Inspection

Consider a drone tasked with inspecting a building. The building's blueprint provides its CoordinateABI: a set of coordinates defining its walls, doors, and windows in a fixed world frame.

- **Navigation Task:** The drone's navigation stack receives a waypoint to fly to a specific door. This waypoint is a CoordinateABI (`world_x=100, world_y=5, world_z=200`). The drone plans a path to this exact location, using the CoordinateABI to avoid collisions with other static obstacles whose own CoordinateABIs are known.

- **Inspection Task:** Upon reaching the vicinity of the door, the drone's onboard cameras begin a detailed inspection. The View Address Generator computes the ViewAddressABI for the door from the drone's current flying position. If the drone hovers to the side, the ViewAddressABI will indicate `visible_face = "side_wall"`. If it rotates to face the door handle, the ViewAddressABI updates to `visible_face = "front_door"`.
- **AI Perception:** An AI analyzing the video feed from the drone's cameras would consume the ViewAddressABI to understand the context of each frame. Seeing `visible_face = "front_door"` allows the AI to know it is looking at the door's primary feature, which is critical for tasks like recognizing the handle or knocking on it. The AI's reasoning is anchored to the perceptual truth of the moment, which is derived from the invariant spatial truth of the building's CoordinateABI.

This use case highlights how the separation allows different systems to work with the appropriate level of abstraction. The flight controller works with absolute coordinates for safety and efficiency, while the inspection AI works with relative perceptions to perform its task. Merging these concepts would create a fragile system; for example, a navigation algorithm that considers a door's "openness" based on its visible face would fail if the camera angle changed, whereas a system that reasons about the door's actual hinge mechanism (part of its CoordinateABI) would be robust.

Scenario 3: Multi-Agent Interaction and Conflict Resolution

In a multiplayer game or a complex simulation with many agents, spatial ownership becomes critical. Imagine two players trying to claim the same resource node.

- **Resource Definition:** The resource node is defined by a `CoordinateABI` at `world_x=50, world_y=10, world_z=50`. It also has a `spatial_ownership` field, initially unclaimed.
- **Player 1 Action:** Player 1 moves their character to the node's coordinates. The game logic checks the CoordinateABI and grants ownership to Player 1. The `spatial_ownership` field is updated.
- **Player 2 Action:** Player 2 moves to the same coordinates. The game logic again consults the CoordinateABI and sees that the node is already owned, denying Player 2's claim.
- **Perceptual Discrepancy:** Crucially, the outcome of this interaction is determined by the invariant CoordinateABI, not by what either player *sees*. Player 1 might be seeing the top of the node, giving a `ViewAddressABI` of `visible_face="top"`. Player 2 might be approaching from a hill and only sees the node's side, giving a `ViewAddressABI` of `visible_face="side"`. Despite this difference in perception,

the logical outcome (who owns the node) is identical for both. If the ownership rule were based on `visible_face`, it would be nonsensical and prone to exploitation.

This scenario underscores the necessity of the separation. High-level logic governing fairness, rules, and state transitions must operate on the ground-truth spatial state provided by the CoordinateABI. Allowing perception to influence such logic would lead to an unstable and unpredictable system. The ViewAddressABI is for presentation and for informing an agent's *view* of the world, not for changing the world's *state*.

Strategic Implications: Preventing Architectural Contamination and Ensuring System Integrity

The formalization of a strict boundary between CoordinateABI and ViewAddressABI is not merely an incremental improvement in software design; it is a strategic imperative for ensuring the long-term integrity, scalability, and maintainability of complex semantic runtime systems. The primary strategic benefit is the prevention of "architectural contamination," a condition where modules become tightly coupled through the sharing of mixed-state information, leading to fragility, increased complexity, and a high cost of change. By enforcing a clear separation of concerns, this architecture establishes a robust foundation that allows different parts of the system to evolve independently.

Architectural contamination arises when a module designed for one purpose begins to rely on the internal state or implementation details of another module operating on a different abstraction level. For example, if the rendering pipeline were to directly query an object's position for physics calculations, or if the AI navigation logic depended on the camera's field of view to determine a valid path, the entire system would become brittle. A change in the rendering pipeline's coordinate transformation logic could inadvertently break the physics simulation, and a modification to the AI's pathfinding algorithm could require rewrites in the camera system. The separation enforced by the ABIs mitigates this risk by creating well-defined interfaces. The Physics Engine's contract is to receive CoordinateABIs. The Renderer's contract is to receive ViewAddressABIs. As long as these contracts are honored, the internal implementations of the modules can change freely without affecting the others. The View Address Generator becomes the single point of responsibility for the translation logic, isolating it from the rest of the system.

This architecture promotes modularity and composability, which are essential for building large-scale systems. Different teams or components can be developed in parallel,

each working against their respective ABI contracts. A team developing a new rendering technology can focus on consuming the ViewAddressABI, knowing that it will always receive the correct perceptual information derived from a stable spatial foundation. Similarly, a team working on a new physics model can optimize the CoordinateABI processing without worrying about how that affects the visual presentation. This aligns with principles from formal methods in software engineering, such as the Agent Behavioral Contracts (ABC) framework, which emphasizes interface compatibility and assumption discharge to ensure that individual components can be composed into a reliable whole [11]. The ABC framework's reliance on a clear boundary between contract logic and feature extraction layers is mirrored in the relationship between the View Address Generator (which extracts perceptual features) and the rendering or AI modules (which consume the resulting contract).

From a practical standpoint, this architecture simplifies debugging and testing. When a bug occurs, the first question is always, "Which space are we in?" [12]. In a contaminated system, this question is often unanswerable, leading to hours of fruitless investigation. With the proposed ABIs, the answer is always clear. If a rendering artifact appears, the issue lies within the View Address Generator or the renderer. If a physics collision fails, the issue lies within the physics engine or the data feeding the CoordinateABI. This clear division of responsibility drastically reduces the search space for bugs. Furthermore, testing becomes more straightforward. Unit tests for the physics engine can be written against pure CoordinateABI inputs. Unit tests for the View Address Generator can verify that it correctly translates a given CoordinateABI and observer state into the expected ViewAddressABI under various conditions (e.g., different angles, distances, and occlusions).

In conclusion, the research goal of formalizing the separation between CoordinateABI and ViewAddressABI represents a shift from ad-hoc, interconnected designs to a principled, layered architecture. The theoretical justification for this separation is strong, drawing parallels to foundational concepts in computer graphics, robotics, GIS, and cognitive science. By defining precise, compiler-style interface specifications, the architecture enforces a clean boundary that prevents the dangerous mixing of spatial and perceptual concerns. The dependency flow—from invariant spatial truth to variable perceptual truth—creates a robust data pipeline that supports modular, scalable, and maintainable system development. Through use-case scenarios, we have demonstrated how this architecture allows a single, unchanging world state to yield a multitude of accurate and context-appropriate perceptions. Ultimately, this approach treats the problem of complexity not by ignoring it, but by systematically organizing it. It acknowledges that a renderer is not reality and a camera is not truth, and builds a system

that respects this fundamental distinction, thereby ensuring the long-term health and integrity of the semantic runtime.

Reference

1. [PDF] API without Secrets: Introduction to Vulkan* Preface https://www.intel.cn/content/dam/develop/external/us/en/documents/api-without-secrets-introduction-to-vulkan-621872.pdf
2. (PDF) Visual Depth Perception of 3D CAD Models in Desktop and ... https://www.researchgate.net/publication/287460049_Visual_Depth_Perception_of_3D_CAD_Models_in_Desktop_and_Immersive_Virtual_Environments
3. CReFT-CAD: Boosting Orthographic Projection Reasoning ... - arXiv https://arxiv.org/html/2506.00568v2
4. 许威威 - 浙江大学计算机辅助设计与图形系统全国重点实验室 http://www.cad.zju.edu.cn/home/weiweixu/
5. Design and Validation of a Virtual Reality Mental Rotation Test https://dl.acm.org/doi/10.1145/3626238
6. OpenGL Voxel engine Face Merging vs Per-Vertex Ambient occlusion https://stackoverflow.com/questions/39350250/opengl-voxel-engine-face-merging-vs-per-vertex-ambient-occlusion
7. Visual Guide for AO and BentNormal Maps - Epic Games Developers https://dev.epicgames.com/community/learning/tutorials/9Gr9/unreal-engine-visual-guide-for-ao-and-bentnormal-maps?locale=zh-cn
8. Voxel Engine Block Placement - Stack Overflow https://stackoverflow.com/questions/76174013/voxel-engine-block-placement
9. ISO 19107:2019(en), Geographic information — Spatial schema https://www.iso.org/obp/ui/es/#!iso:std:66175:en
10. Relationship of OGC Abstract Specification Topics Topic 1... https://www.researchgate.net/figure/Relationship-of-OGC-Abstract-Specification-Topics-Topic-1-provides-the-geometry-data_fig3_44083207
11. Agent Behavioral Contracts: Formal Specification and Runtime ... https://arxiv.org/html/2602.22302v1

12. World Space vs Screen Space in Computer Graphics - LinkedIn https://www.linkedin.com/posts/carcruz_computergraphics-graphicsprogramming-opengl-activity-7413895431958499328-DMQ_
13. Reasoning Path and Latent State Analysis for Multi-view Visual ... https://arxiv.org/html/2512.02340v1
14. Assessing Comprehensive Spatial Ability and Specific Attributes ... https://pmc.ncbi.nlm.nih.gov/articles/PMC12565372/
15. An Empirical Analysis on Spatial Reasoning Capabilities of Large ... https://aclanthology.org/2024.emnlp-main.1195/
16. [PDF] Game Programming Gems 7 https://gaoyichao.com/Xiaotu//papers/Game%20Programming%20Gems%207.pdf
17. RHI_图形API对比(Vulkan、DirectX 12/11、Metal - CSDN博客 https://blog.csdn.net/qq_35312463/article/details/128435825
18. Unity新版本下载-Unity稳定版本 https://unity.cn/release-notes/full/2023/2023.1.0f1
19. i.MX Graphics User's Guide Overview | PDF - Scribd https://www.scribd.com/document/806832229/i-MX-Graphics-User-s-Guide
20. CVPR 2025 Accepted Papers https://cvpr.thecvf.com/Conferences/2025/AcceptedPapers
21. Computer Vision and Pattern Recognition - arXiv https://arxiv.org/list/cs.CV/new
22. Integrating CAD and Orthographic Projection in Descriptive ... - MDPI https://www.mdpi.com/2227-7102/15/11/1492
23. Tf2 — ROS 2 documentation documentation https://ros.ncnynl.com/en/jazzy/Concepts/Intermediate/About-Tf2.html
24. Access the tf Transformation Tree in ROS - MATLAB & Simulink https://ww2.mathworks.cn/help/ros/ug/access-the-tf-transformation-tree-in-ros.html
25. [ROS2基础] TF2使用细节 - 知乎专栏 https://zhuanlan.zhihu.com/p/524590694
26. ROS2 Robot Ground Truth Pose publish - Isaac Sim https://forums.developer.nvidia.com/t/ros2-robot-ground-truth-pose-publish/230508
27. CityGML 3.0: New Functions Open Up New Applications https://www.researchgate.net/publication/339512064_CityGML_30_New_Functions_Open_Up_New_Applications
28. Project voxels from camera space into camera view to get correct ... https://stackoverflow.com/questions/42984674/project-voxels-from-camera-space-into-camera-view-to-get-correct-image-coordinat
29. [PDF] Voxel Map for Visual SLAM - arXiv https://arxiv.org/pdf/2003.02247

30. Multiple View Geometry in Computer Vision: | Guide books https://dl.acm.org/doi/book/10.5555/861369
31. Research on 3D Rendering Technology to Enhance the Spatial ... https://www.researchgate.net/publication/395759680_Research_on_3D_Rendering_Technology_to_Enhance_the_Spatial_Expression_of_Virtual_Art_in_the_Age_of_Digital_Media
32. ROS 2-Based LiDAR Perception Framework for Mobile Robots in ... https://arxiv.org/html/2604.02109v1
33. Building a Perception Pipeline - ROS Industrial Training https://industrial-training-master.readthedocs.io/en/humble/_source/session5/Building-a-Perception-Pipeline.html
34. ROS 2 Cameras - Isaac Sim Documentation https://docs.isaacsim.omniverse.nvidia.com/latest/ros2_tutorials/tutorial_ros2_camera.html
35. Robot Operating System 2 (ROS2)-Based Frameworks for ... - MDPI https://www.mdpi.com/2076-3417/13/23/12796?type=check_update&version=1
36. osmAG-Nav: A Hierarchical Semantic Topometric Navigation Stack ... https://arxiv.org/html/2603.28271v1
37. (PDF) GLASS: Geometry-aware Local Alignment and Structure ... https://www.researchgate.net/publication/402863973_GLASS_Geometry-aware_Local_Alignment_and_Structure_Synchronization_Network_for_2D-3D_Registration
38. Computational Geometry: Algorithms and Applications | Guide books https://dl.acm.org/doi/10.5555/1370949
39. [PDF] 2026 SJTU Summer Research Internship Program https://global.sjtu.edu.cn/summer-program/images/2026%20SJTU%20SUMMER%20RESEARCH%20INTERNSHIP%20PROGRAM.pdf
40. ISO 19107 Geographic information — Spatial schema https://committee.iso.org/sites/tc211/home/projects/projects---complete-list/iso-19107.html
41. Unreal Engine 5.0 Release Notes - Epic Games Developers https://dev.epicgames.com/documentation/unreal-engine/unreal-engine-5.0-release-notes?application_version=5.0&lgNYsuL=UjWabWGy1Cn
42. Enhanced Perception for Autonomous Driving Using Semantic and ... https://pmc.ncbi.nlm.nih.gov/articles/PMC9269728/
43. A Survey on Occupancy Perception for Autonomous Driving - arXiv https://arxiv.org/html/2405.05173v1
44. PesRec: A parametric estimation method for indoor semantic scene ... https://www.sciencedirect.com/science/article/pii/S1569843224004898

45. Integrated design-sense-plan architecture for autonomous ... https://www.frontiersin.org/journals/robotics-and-ai/articles/10.3389/frobt.2022.911974/full

46. CityGML 3.0: New Functions Open Up New Applications | PFG https://link.springer.com/article/10.1007/s41064-020-00095-z

47. CityGML in the Integration of BIM and the GIS: Challenges ... - MDPI https://www.mdpi.com/2075-5309/13/7/1758

48. https://zenodo.org/records/6000983/files/Custom-La... https://zenodo.org/records/6000983/files/Custom-Labels.csv?download=1

49. (PDF) A Java Implementation of the OpenGIS™ Feature Geometry ... https://www.researchgate.net/publication/228725565_A_Java_Implementation_of_the_OpenGIS_Feature_Geometry_Abstract_Specification_ISO_19107_Spatial_Schema

50. OGC and ISO Compliance https://docs.oracle.com/en/database/oracle///oracle-database/26/spatl/ogc-and-iso-compliance.html

51. How to do face culling in a voxel game using Java and LWJGL https://stackoverflow.com/questions/77870072/how-to-do-face-culling-in-a-voxel-game-using-java-and-lwjgl

52. Nanite Voxel 全流程解析 - 知乎专栏 https://zhuanlan.zhihu.com/p/1962121162306846748

53. Enhanced Perception for Autonomous Driving Using Semantic and ... https://www.mdpi.com/1424-8220/22/13/5061

54. Gau-Occ: Geometry-Completed Gaussians for Multi-Modal 3D ... https://www.researchgate.net/publication/403112161_Gau-Occ_Geometry-Completed_Gaussians_for_Multi-Modal_3D_Occupancy_Prediction

55. Measuing lag between Isaac sim and ros2 - NVIDIA Developer Forums https://forums.developer.nvidia.com/t/measuing-lag-between-isaac-sim-and-ros2/294562

56. Why is the TF tree structure in Isaac Sim different from standard ROS ... https://forums.developer.nvidia.com/t/why-is-the-tf-tree-structure-in-isaac-sim-different-from-standard-ros-2-do-i-have-to-follow-isaac-sims-structure/332331

57. ROS 2 101，第12章空间直觉：TF2 坐标变换树+ RViz2 - 知乎专栏 https://zhuanlan.zhihu.com/p/2029433757338191698

58. OGC CityGML Quality Interoperability Experiment - ResearchGate https://www.researchgate.net/publication/310501677_OGC_CityGML_Quality_Interoperability_Experiment

59. DOTS versus Nanite? - Unity Discussions https://discussions.unity.com/t/dots-versus-nanite/790763

60. Unity's DOTS vs Unreal engine - questions - Cinematics & Media https://forums.unrealengine.com/t/unitys-dots-vs-unreal-engine-questions/227603
61. UnrealFest2024 UE5中TSR、Nanite、Lumen、VSM图形功能剖析 https://zhuanlan.zhihu.com/p/1979686024402708097
62. ECS for Unity https://unity.com/ecs